

certain events (like triggering a workflow only when a push event occurs on the main branch).

Followed by this is a ‘jobs’ key that defines all the jobs in the workflow, with a name for each job. In our example, there is only one job named ‘build’ that runs on its own virtual machine runner with the *ubuntu-latest* image as defined by the *runs-on* key.

Within the job is a list of steps that need to run in sequence on the same virtual machine in order to accomplish the task of the job. In our example, these might be to do whatever is required in order to create a build.

In Figure 3, the first step uses an ‘action’ from the GitHub Marketplace that does a check of the code; the next two steps run scripts within the shell of the virtual machine runner, and the last step runs a custom action that’s defined in the repository. You can use any combination of these scripts and actions necessary to accomplish the tasks for your job.

Why use GitHub Actions?

GitHub Actions is native to GitHub and integrates well with the common software development workflows you might be familiar with on it. You can accomplish many of your automation needs solely with GitHub Actions, or even combine them with other tools to manage your workflows in a seamless way.

Actions is independent of the language/framework that you use for your project, and you can use it to automate any of your workflows. Whether you want to run simple linting using JS libraries, create a distributable from your code, build a container image, or do anything else in between, you can accomplish it using GitHub Actions.

Since workflow jobs run on virtual machine runners, you can run your workflows on Linux, macOS, Windows or even ARM. You can



Figure 2: Example of push and build

```

1 name: CI
2
3 on:
4   push:
5     branches: [ main ]
6   pull_request:
7     branches: [ main ]
8   workflow_dispatch:
9
10 jobs:
11   build:
12     runs-on: ubuntu-latest
13
14     steps:
15
16     - name: Using the Checkout action from Marketplace
17       uses: actions/checkout@v2
18
19     - name: Run a one-line script
20       run: echo Hello, world!
21
22     - name: Run a multi-line script
23       run: |
24         echo Add other actions to build,
25         echo test, and deploy your project.
26
27     - name: My own action in the same repo
28       uses: ./
29

```

Figure 3: Example workflow file

choose which architecture and OS you want for each of your jobs within your workflow, based on your own use case. These virtual machine runners can be hosted by GitHub, or you can even self-host them. Alternatively, you can choose to skip the virtual machine runner altogether and just run a container image when your workflow gets triggered.

GitHub Actions is used by many developers, open source projects and businesses for their automation needs. If you check out various open source projects on GitHub, you might find that many of them are using Actions for their project management, CI/CD pipelines or other automations. There are more than 12,000 community-powered Actions extensions on the